Accelerating DNN Communications by Hierarchical Resharding

Ruben COLL SANCHEZ[1,4], Hongxing WANG[1], Hector BUFFIÈRE[1,3], Pierre LECA[1], Teng SU[2] and Chong LI[1*]

[1]Huawei Paris Research Center, Boulogne-Billancourt, France.
[2]Huawei Technologies Co., Ltd, Hangzhou, China.
[3]École Normale Supérieure, CNRS, PSL University, Paris, France.
[4]Sorbonne Université, Paris, France.

*Corresponding author(s). E-mail(s): ch.l@huawei.com;

Abstract

High-level deep learning frameworks have been proposed to simplify the deployment of deep learning models, including their parallelization on a given machine. Meanwhile, modern parallel machines leverage different types and levels of communication to interconnect devices in order to maximize their computing power. We formalize data redistribution costs and propose an algorithm that efficiently schedules data redistributions on hierarchical parallel machines, so that large data transfers take place between closely connected devices and only small ones traverse slower links. Our approach is implemented in the HyperParallel open-source framework and applied to the end-to-end training of three state-of-the-art large language models on up to 128 accelerators, where we observe speedups of up to 1.204x in training step time.

Keywords: Parallel Programming, Cost Model, Hierarchical Communication, Distributed Deep Learning, Performance Optimization, Collective Communication.

1 Introduction

In recent years, the appearance of giant deep learning (DL) models [1–4] leads to a dramatic demand for computing power. On one hand, more and more

tensor-oriented large-scale parallel computers have been designed. Modern clusters for AI computing, like Google Ironwood [5], Huawei CloudMatrix384 [6] or Nvidia Vera Rubin NVL72 [7] contain hundreds of general-purpose computing units and accelerators. These devices are connected with high-bandwidth buses like NVLink [8] or UnifiedBus [9] forming computing nodes that follow a certain topological design. In turn, nodes are assembled into racks, and racks are stacked into computing clusters. The hierarchical design allows manufacturers to scale up and scale out their products in an independent manner while keeping high-performance.

On the other hand, in the domain of parallel programming, different kinds of high-level parallelisation techniques also emerged in order to reduce the execution time of such Deep Neural Network (DNN) models. These techniques were generalized into different deep learning parallelism paradigms, such as data parallelism (splitting batches into mini-batches which are given to different devices) [10] and operator-level model parallelism (splitting parameters into different devices) [11]. Different parallel strategies can be again combined at the same time in a hybrid strategy for parallelizing the same operator.

Modern AI frameworks like PyTorch [12], TensorFlow [13] or MindSpore [14] exhibit a modular design, which allows DNN programmers to define their networks as operator graphs acting on tensors in a high-level manner. The modularity allows better portability, as different parts of the framework can be independently modified and optimized depending on e.g. the hardware or the type of network. The top-level module is the python interface where DL engineers can concisely define various neural networks. One mid-level module of the framework can then automatically split the computation depending on the number of devices used, and a second one assigns each part of the workload to devices of the cluster and connects them with necessary logical communications. The bottom-level modules use communication libraries to write the final machine code. Our work belongs to the second mid-level module and seeks to provide new optimizations orthogonally to the already very successful parallel techniques mentioned above.

However, the choice of high-level strategies of parallelism is still a complex problem. The number of possible strategies that can be chosen depends on the number of operators and the number of devices. The best strategy for a given operator also depends on the strategy of its neighbors when redistributions are taken into account. To avoid increasing this complexity, strategies are often chosen with a simplified view of the implied redistributions. For example, by not considering the precise low-level device assignment of tensor shards, nor the precise redistribution operations that need to be performed. With DL relying mainly on linear algebra operations that are inherently highly parallelizable, and communication being much slower than computation, almost all of the extra cost induced by distributed training comes from redistributions, as showcased in [15]. As hybrid strategies are more commonly used nowadays in order to find a balance between speed and memory requirement, this causes

more frequent and more costly redistributions, amplifying the performance issues caused by inefficient choices of redistributions.

In this paper, we formalize the redistribution costs among different parallel strategy configurations, and propose a novel algorithm for redistributing tensor shards between two operators with different device assignments. The key idea behind this algorithm is to favor performing large data transfers between devices with faster communication links and small data transfers between devices with slower communication links. In this way, we exploit the properties of the hierarchical nature of clusters, making appropriate assumptions to propose a polynomial heuristic algorithm to a NP-hard problem.

The paper is organized as follows: Sec. 2 introduces how a DNN is partitioned to be executed in parallel; Sec. 3 explains how such parallel execution is deployed on a distributed cluster; the main ideas are in Sec. 4 which presents our distributed cost analysis and algorithm to enhance the performance of the above distributed execution; Sec. 5 shows our experiment results; Sec. 6 discusses recent related works; and Sec. 7 concludes the paper.

2 Parallelizing Deep Neural Networks

A DNN can be represented as a computational graph $G = (V, E)$ where V is a set of operators, and E is a set of directed edges that encode the data dependencies between computations. An operator is defined as a pair (o, t) where o describes the type of the operator (*Add*, *MatMul*, ...) and $t = (t^{\text{in}}, t^{\text{out}})$ is a pair of lists of tuples of integers that represent the input and output tensors' shapes, respectively. The lists could be empty, as in the case for tensor loads and stores. When t^{in} or t^{out} only contain one element, we will identify them with the element itself to simplify the notation. The above defines the basic semantics of a deep learning model from a computational point of view.

Computational graphs can be executed in parallel by partitioning their operators on different devices. A fraction, or *shard*, of the input tensors is assigned to each device, which computes a partial operator to obtain the corresponding *shard* of the output tensor. The parallelization process is generally divided in 3 phases:

1. A *sharding strategy* is assigned to each operator, specifying how to cut the input and output tensors and divide their data layout.
2. The compiler uses the sharding strategy to divide the input tensors in shards, and assigns them to devices.
3. The compiler generates and inserts data redistribution operations, if further operators depend on the result, but their sharding strategy is not compatible.

Sharding can be defined by a map $\mathcal{S} : V \to \tilde{V}$ where each vertex of $\tilde{V}$ is now a triple $\mathcal{S}((o, t)) = (o, t, s)$. The strategy $s = (s^{\text{in}}, s^{\text{out}})$ is also a pair of lists of tuples of integers, which now specify the sharding of each input and output tensors' dimensions in an integer number of splits. For instance,

an input tensor of shape $t^{\text{in}} = (16, 1024, 1024)$ could be assigned a strategy $s^{\text{in}} = [\![4, 1, 2]\!]$. The first tensor dimension would be split 4 times and the last would be split twice. The tensor shards would then be of shapes $(4, 1024, 512)$, and be evenly spread into $4 \times 1 \times 2 = 8$ partitions. Note that 8 partitions could be deployed on 8 but also on 16, on 32, ... devices, for instance by replicating partition number i on all devices of index i modulo 8. An example of the distributed computation of an Add operator with two input tensors and one output tensor is shown in Figure 1.

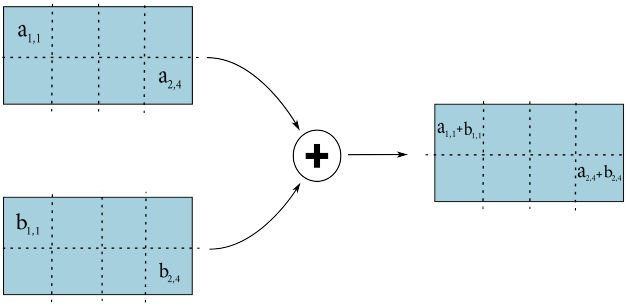


Fig. 1 Example of an Add operator computing $c = a + b$ with input and output tensors sharded 8 times with strategies $[\![2, 4]\!]$. This computation can be run on $N \geq 8$ devices, each tensor shard being replicated $N/8$ times.

Most deep learning frameworks use sharding strategies as the unique way to define DNN partitioning, and hide the rest of the details from the user. We argue that this is an oversimplification, since the redistributions do not only depend on strategies, but also on the distribution of shards among devices. Detailing how strategies are translated into device assignments is crucial to get an idea of the communication cost of redistributions.

Example 1. *Let us consider the situation depicted in Figure 1, supposing that 8 abstract parallel devices* $(d_i)_{i \leq 8}$ *are used to compute* $c_{i,j} = a_{i,j} + b_{i,j}$*, where* $1 \leq i \leq 2$ *and* $1 \leq j \leq 4$*. Let us assume that* a *and* b *are the result of Load operators, whose output strategy is also* $[\![2, 4]\!]$*. Of all possible* $(8!)^2$ *assignments of input shards to devices, the only cases where no data redistribution needs to be inserted are those in which* $a_{i,j}$ *and* $b_{i,j}$ *are assigned to the same device, for each pair* i, j*. That amounts to a total of* $8!$ *assignments. Otherwise, data needs to be transferred between devices so that the final output is consistent with the sequential computation.*

The assignment problem of the example is straightforward to solve, by keeping track of the symmetries of the inputs and an order of the devices. It is much more difficult to generate assignments in the general case: real case scenarios include computational graphs with hundreds or thousands of operators, and complex dependency patterns. The problem can be reduced

to the multiprocessors makespan minimization problem with precedence constraints and communication overhead, which is NP hard [16]. Assignments and redistributions are often generated using heuristics and reducing the space of possibilities to a tractable size. We thus propose to shift the threshold of DNN parallelization design to include them as part of the formalization of the process.

3 Distributed Deep Learning

In order to satisfy the rising demand of computing power that deep learning workloads require, many architectural patterns have been proposed to improve the efficiency of high-performance systems. Structuring parallel computers is key to simplifying the mapping of tensor computations on the hardware.

The use of purpose-specific hardware accelerators has been successful: in the last years, the market has been dominated by graphic, tensor and neural processing units equipped with cube units for matricial operations and high-bandwidth buses to handle large amounts of data. By scaling up these units and increasing their individual capabilities, it is possible to automatically enhance performance globally. However, meeting the needs of large deep learning models by increasing the raw computing power is impossible.

A complementary approach consists in aggregating multiple, possibly heterogeneous units into a network that combines each individual worker to increase the total throughput of the system. The *topology* of a computing network is the logical design that organizes the communication between devices, as nodes of a graph. In the domain of distributed deep learning, hierarchical heterogeneous topologies have become the design of choice of many manufacturers [5–7]. These topologies organize groups of devices in a tree-like structure, where devices of different nature stand in different levels of the hierarchy. Computing units lie in the lowest levels, while switches and interconnection devices organize the medium and upper levels. Each node of the same level has the same number of children nodes. Hierarchical topologies are easy to upgrade, analyze and combine, and are particularly suited for processing large amounts of data, since it is possible to multiply the computing power of the system by modifying the number of nodes at a certain level.

Example 2. *Huawei CloudMatrix384 is a next-generation AI datacenter architecture that integrates up to 384 Ascend 910C NPUs, divided into 48 nodes of 8 accelerators, with 2 Ascend 910C computing dies each [6]. This architecture features a leaf-spine topology, or Clos topology [17]. Inside an accelerator, the two Ascend 910C dies communicate through ultra-high-bandwidth NIC. At each node, these 16 computing units are connected to seven leaf switches through Unified-Bus links [9], and each node's switches are connected to seven planes of spine switches. As a hierarchical machine, the spine switches sit at the top level of the hierarchy, the leaf switches stand in the medium level and the lower level contains the computing units.*

Any hierarchical computing system with N computing units can be abstracted into a set of device dimensions represented by a *device matrix*: a vector d where d_i is the number of children of each node in the i-th level of the hierarchy. As a consequence $\prod_i d_i = N$. The idea that directs most of the parallel strategies for deep neural networks consists of assigning shard dimensions to device dimensions, to keep data layouts consistent across the hierarchy.

A tensor dimension i can be sharded along device dimension j if it can be split equally between the corresponding nodes in the hierarchy. Since each node of the tree leads to the same number of leaf workers, this is equivalent to checking if d_j divides t_i^{in}. The assignment of shard parallelism dimensions to device dimensions is encoded by the map $\mathcal{M} : \tilde{V} \to \hat{V}, (o, t, s) \mapsto (o, t, s, m)$ where $m = (m^{\text{in}}, m^{\text{out}})$ is a pair of *tensor maps* with the same length and tuple shapes as t and s. If the input (output) tensor dimension i is not sharded, that is $s_i^{\text{in}} = 1$ ($s_i^{\text{out}} = 1$), then $m_i^{\text{in}} = \varnothing$ ($m_i^{\text{out}} = \varnothing$). Otherwise, the input (output) tensor dimension i is sharded along the device dimension m_i^{in} (m_i^{out}).

Data parallelism for deep neural networks [10] exploits this idea by sharding the tensor dimension corresponding to the batch size across the lowest level of the hierarchy, and letting the dimensions of the model tensors unsharded. Extensions of this paradigm, like fully-sharded data parallelism (FSDP) [18] or hybrid-sharded data parallelism (HSDP) [19], shard also the model parameters in the lowest or in intermediate levels of the hierarchy, respectively. Expert parallelism, a popular flavor of model parallelism, focuses on parallelizing Mixture-of-Experts (MoE) [20] networks by sharding the expert dimension. Sequence parallelism shards the sequence length dimension in language models at specific operators [21].

Redistributions are needed when the tensor maps of a sharding assignment differ. The data movement is usually expressed through collective communication operations (*AllGather*, *ReduceScatter*, *AllToAll*...) that act on different levels of the hierarchy, and define concisely the dataflow [22]. The same redistribution can be achieved with different sequences of collective communication operations, with vastly different execution times. With larger topologies, and more diverse tensor shapes, the number of combinations increases. Using sophisticated tensor layouts with large numbers of devices is becoming the norm with automatic fine tuning of sharding strategies to improve large scale parallel training performance. Hence, we propose in the following section a topology-aware method to optimize the generation of hierarchical collective communication operation sequences and analyze its performance from a symbolic point of view.

4 Performance Enhancement

Our algorithm takes an input tensor map m^{in} and the output tensor map m^{out} to which it must be converted. It returns a sequence of redistribution operations that transform the input tensor map into the output tensor map, minimizing the communication cost. This algorithm can be applied separately

on each operator of the deep neural network, since the choices of communication operations do not interfere with each other. The communication overhead introduced by a parallelization strategy can be quantified by analyzing the set of redistributions that our proposed method obtains at the different operators of the network. Before formally presenting our contribution, we clarify first some preliminaries.

4.1 Cost model

We use the SGL bridging model [23] as a high-level framework to reason about the performance of parallel programs. SGL represents distributed systems as a hierarchical abstract machine organized as a rooted tree. The leaves of the tree represent workers that perform computations, and inner nodes represent masters that interconnect their children and channel the communications between them. Each master at depth level i in the tree has the same amount of children. This structure is coherent with the design of modern AI clusters, and thus we find it appropriate to ground our arguments on it. However, any other modelling of hierarchical computations should be compatible with our contributions.

In SGL, all groups of devices at depth level i of the tree share the same communication capacity g_i. The cost of the communication between two devices associated with leaves u and v is $q \times g_i$, where q is the amount of data transferred and i is the level of the lowest common group, or ancestor of u and v in the tree. The cost of a sequence of communications is obtained as the sum of the costs of each individual communication operation. This does not take into account overlap between computation and communication, which is a limitation of our work. The communication capabilities of the master nodes are assumed to be increasing the deeper they are in the tree. We also assume that $g_i \ggg g_{i+1}$, which in practice means that we can always consider optimal to minimize the amount of communication going through a slower connection at the expense of augmenting the amount of communication going through faster connections, even by a bigger amount. This is the basis of our analysis on which the choices of our algorithm depend. We observed that the bigger a cluster is, the more benefit we draw from this strongly hierarchical model, as the communication links between faraway devices are orders of magnitude slower than the ones connecting close devices.

4.2 Collective communication

The two collectives used in our analysis are *AllGather* and *Slice*. These are available on all hardware architectures and span all possible redistribution sequences, as will be shown below. We do not assume any specific implementation of them: further adjustments can orthogonally be applied in particular cases to improve the overall performance of the redistributions. Some advanced operations like *AllToAll* could also be considered, but were left out to simplify the analysis. They are non-primitive collective communications, and require

case-by-case optimal implementations [24]. Our algorithm focuses on general cases, in order to be applicable in the widest range of configurations. Taking into account a more diverse set of primitives could be a ground for future work.

AllGather is a collective communication operation that concatenates tensor slices from all devices in a device group and stores the result on each device. If applied along a tensor dimension i sharded along device dimension m_i^{in} (written $AllGather(i)$), the output tensor map now has $m_i^{\text{out}} = \varnothing$ and the communication cost of the operation is $g_{m_i} \times \prod_k \frac{t_k}{s_k}$ (the second factor represents the size of the tensor initially on each device). The tensor shards stored on each device have their size multiplied by d_{m_i} at the end of the operation.

Slice is the opposite operation. It does not incur any data communication; instead, it splits tensors in order to distribute them between devices of a group. When applied along a tensor dimension i and device dimension j (written $Slice(i, j)$), if $m_i^{\text{in}} = \varnothing$ and $m_k^{\text{in}} \neq j$ for all k, the output tensor map then has $m_i^{\text{out}} = j$. Since it is executed locally on each device there is no communication cost. The tensor shards stored on each device have their size divided by d_j after the operation.

4.3 Redistribution chains

For most input/output tensor maps, redistributions will be straightforward. Some tensor dimensions i can satisfy $m_i^{\text{in}} = m_i^{\text{out}}$, in which case nothing is to be done. If $m_i^{\text{in}} \neq \varnothing$ and $m_i^{\text{out}} = \varnothing$, an $AllGather(i)$ needs to be performed so that the tensor dimension i goes from sharded to replicated. In the general case, any device dimension can appear in the input tensor map or the output tensor map. If it appears only at the input tensor map, data will need to be gathered and concatenated across devices; if it only appears at the output, then data will be split. Those device dimensions appearing in both tensor maps induce a permutation of the underlying data. Every permutation can be decomposed into a product of disjoint cycles that can be applied in any order (that is, they commute). Efficient redistributions handle data concatenations, splits and cycles in a sequence that minimizes the communication volume.

Example 3. *Let* $m^{\text{in}} = [1\ 2\ 3]$ *and* $m^{\text{out}} = [2\ 1\ \varnothing]$. *The two first tensors dimensions are sharded in both layouts, but they need to be transposed. The third is only split in the input; it must be gathered across device dimension 3. In order to permute tensor dimensions 1 and 2, data must be gathered at the lowest common level of the hierarchy, and then split again. From our assumptions, concatenating data on multiple device dimensions is optimal if higher (slower) levels are gathered first, so device dimension 1 should be gathered before device dimension 2. If the third tensor dimension is concatenated before the permutation, the communication volume will be increased, and each device will need to store the full tensor to permute data. If not, the permutation will only transpose a shard of the tensor at each intermediate node; after that, the third dimension can be concatenated with an AllGather between them. Thus it is better to permute the first two dimensions, and leave the last at the end.*

An efficient redistribution sequence is (we write AG for AllGather *and S for* Slice*)* $[1\ 2\ 3] \xrightarrow{\text{AG}(1)} [\varnothing\ 2\ 3] \xrightarrow{\text{AG}(2)} [\varnothing\ \varnothing\ 3] \xrightarrow{\text{S}(1,2)} [2\ \varnothing\ 3] \xrightarrow{\text{S}(2,1)} [2\ 1\ 3] \xrightarrow{\text{AG}(3)} [2\ 1\ \varnothing]$. *Steps 1, 3, 5 and 6 are shown in Figure 2.*

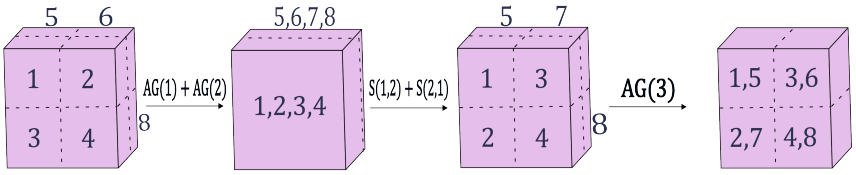


Fig. 2 Redistribution sequence on 8 devices

We list the different types of chains and cycles in Table 1. They are represented with two rows of device dimensions, corresponding to rearranged subsets of the input tensor map and the output tensor map respectively. Each column symbolizes a different tensor dimension, not necessarily in the order of the original tensor shape. Device dimensions with a variable name are assumed to be different from $\varnothing$, two device dimensions of a chain with the same name are equal, and a variable with a tilde on top means that it is not present anywhere else in the tensor maps. Since we assume that the ratios g_i/g_{i+1} are very large, the cost of a sequence of communication operations can be approximated to the cost of the communication that uses the slowest link. It can be shown that the table enumerates all possible chain formats. Indeed, in any redistribution chain, only the first and the last element can be empty. If none are empty, there are two possibilities: either there is a cycle or not. If only the first or the last is empty, there cannot be a cycle, since the number of non-empty device dimensions in the input and output tensor maps differ. Finally, if both the first and the last element are empty, the chain has a cycle-like form.

Name	**Chain format**
Empty chain	$\begin{bmatrix}\varnothing\\ \varnothing\end{bmatrix}, \begin{bmatrix}m\\ m\end{bmatrix}$
Neutral chain of type 1	$\begin{bmatrix}\varnothing & m_1 & m_2 & \dots & m_l\\ m_1 & m_2 & \dots & m_l & \varnothing\end{bmatrix}$
Neutral chain of type 2	$\begin{bmatrix}\widetilde{m}_1 & m_2 & m_3 & \dots & m_l\\ m_2 & m_3 & \dots & m_l & \widetilde{m}_{l+1}\end{bmatrix}$
Split chain	$\begin{bmatrix}\varnothing & m_1 & m_2 & \dots & m_l\\ m_1 & m_2 & \dots & m_l & \widetilde{m}_{l+1}\end{bmatrix}$
Concatenation chain	$\begin{bmatrix}\widetilde{m}_1 & m_2 & m_3 & \dots & m_l\\ m_2 & m_3 & \dots & m_l & \varnothing\end{bmatrix}$
Cycle	$\begin{bmatrix}m_1 & m_2 & \dots & m_l\\ m_2 & \dots & m_l & m_1\end{bmatrix}$

Table 1 The classification of all redistribution chains, described through tensor maps and device dimensions.

4.4 Main algorithm

We first outline here the main algorithm, and then explain in more details how each redistribution chain is handled. Given the specifics of the *AllGather* and *Slice* operations explicated in 4.2, the communication cost of resharding, associated to the *AllGather* operations, are proportional to the size of the tensor on which they are applied. Since ratios g_i/g_{i+1} are assumed to be very large, we will consider a redistribution to be optimal if its *AllGather* operations along the slowest links are performed on the smallest possible tensors.

In order to achieve this goal, the algorithm handles first the redistribution chains that reduce the sizes of the tensors, then the ones which do not change them, and lastly those that increase them. Among the chains that decrease the size of the tensors, it computes those that need an *AllGather* with the fastest links first, as they will be performed on larger tensors. Conversely among the chains that increase the size of the tensors those that need to use an *AllGather* on a slow link are done first. The pseudo code is given in Algorithm 1. The Reshard procedure is a set of handcrafted routines for each different type of chain that intend to minimize the communication cost of the resharding they produce.

We list below how Reshard handles redistribution chains of each type.

- **Empty chain**: nothing needs to be done.
- **Split chain** $\begin{bmatrix} \varnothing & m_1 & m_2 & \dots & m_l \\ m_1 & m_2 & \dots & m_l & \widetilde{m}_{l+1} \end{bmatrix}$: we concatenate along the input device dimension then split along the output device dimension from right to left, which gives the resharding sequence $<$*AllGather*$(l+1)$, *Slice*$(l+1, \widetilde{m}_{l+1})$, *AllGather*$(l)$, *Slice*$(l, m_l)$, ..., *AllGather*$(2)$, *Slice*$(2, m_2)$, *Slice*$(1, m_1) >$.
- **Neutral chains of type 1** $\begin{bmatrix} \varnothing & m_1 & m_2 & \dots & m_l \\ m_1 & m_2 & \dots & m_l & \varnothing \end{bmatrix}$: they need to be treated from left to right in order to avoid doing two consecutive *AllGather*, which gives $<$*AllGather*(2), *Slice*$(1, m_1)$, ..., *AllGather*$(l+1)$, *Slice*$(l, m_l)>$.
- **Neutral chain of type 2** $\begin{bmatrix} \widetilde{m}_1 & m_2 & m_3 & \dots & m_l \\ m_2 & m_3 & \dots & m_l & \widetilde{m}_{l+1} \end{bmatrix}$: we do the same as for Split chains, except the first tensor dimension also needs to be concatenated, which gives the sequence $<$*AllGather*(l), *Slice*$(l, \widetilde{m}_{l+1})$, *AllGather*$(l-1)$, *Slice*$(l-1, m_l)$, ..., *AllGather*(1), *Slice*$(1, m_2) >$.
- **Concatenation chain** $\begin{bmatrix} \widetilde{m}_1 & m_2 & m_3 & \dots & m_l \\ m_2 & m_3 & \dots & m_l & \varnothing \end{bmatrix}$: Here we can only start with an *AllGather*, and by doing so on any dimension leaves us with two neutral chains (of types 1 on the right and 2 on the left), which we can then solve independently as previously. We issue the *AllGather* along the device dimension with the slowest communication capacity.

- **Cycle** $\begin{bmatrix} m_1 & m_2 & \dots & m_l \\ m_2 & \dots & m_l & m_1 \end{bmatrix}$: here again the first operation must be an *All-Gather*, after which it becomes a split chain. We start concatenating by issuing it along the device dimension with the slowest communication capacity.

Algorithm 1 Computes a resharding sequence that minimizes the volume communicated through the slowest links

Input: input/output tensor maps m^{in} and m^{out} of length l
Output: ReshardingSequence

```
chains ← {∅, ∅, ∅}                ▷ Decreasing, keeping, and increasing tensor sizes
for 1 ≤ i ≤ l do
    if i is not in any chain seen yet then
        chain ← ComputeChain(i)
        type ← ChainType(chain)
        chains[type] ← chains[type] ∪ {chain}
    end if
end for
Sort chains[0] by decreasing maximum element        ▷ Lowest levels first
Sort chains[2] by increasing minimum element        ▷ Highest levels first
ReshardingSequence ← ∅
for 0 ≤ type ≤ 2 do
    for all chain in chains[type] do
        ReshardingSequence ← ReshardingSequence + Reshard(chain)
    end for
end for
```

Given that resharding, sorting, and computing chains, obtaining chain types, keeping dictionaries and executing l bounded loops are all polynomial operations in l, the algorithm obtained is polynomial in the length of the tensor maps inputted. Assuming a tolerant bound of $l \leq 20$ for the length of the tensor maps, which ultimately accounts for the shapes of input and output tensors in an operator, we obtain that it is possible to analyze the cost of the redistributions applied at each operator of a DNN in polynomial time.

5 Evaluation

We applied our formalism to analyze and optimize the data redistributions performed as a part of the training of different large-language models. In particular, we focus on disaggregating the supervised training of multi-modal language models through fully-sharded data parallelism (FSDP) [18, 25] on a Huawei CloudMatrix384 cluster [6], using the HyperParallel distributed training framework [19]. For our tests, we use a maximum of 8 nodes of the 48

possible nodes. Because the topology shape is not affected with this restriction, we argue that this does not affect the validity of the experiments.

After identifying the main communication collectives involved, we found that the *AllGather* redistribution used during the parameter unsharding of FSDP does not follow the principle of our algorithm: when increasing the size of the tensor, data is gathered first using the fast links (intra-node communication), and then gathered globally using the slow links (inter-node communication). In terms of redistribution sequences, with device dimension 1 denoting the inter-node level and device dimension 2 the intra-node level, this means that concatenation chains are handled as in $[1\ 2] \xrightarrow{\mathrm{AG}(2)} [1\ \varnothing] \xrightarrow{\mathrm{AG}(1)} [\varnothing\ \varnothing]$, while our algorithm would choose $[1\ 2] \xrightarrow{\mathrm{AG}(1)} [\varnothing\ 2] \xrightarrow{\mathrm{AG}(2)} [\varnothing\ \varnothing]$. Therefore, we designed a hierarchical *AllGather* collective that follows the redistribution sequences obtained with our algorithm. Throughout the rest of the section, we compare its efficiency with respect to the existing implementation.

5.1 Microbenchmark

We implemented the hierarchical version of *AllGather* as a plug-in for HyperParallel[1], using its high-level PyTorch interface. Since we cannot control the use of NIC or intra-node UB in the computing units inside an Ascend 910C accelerator of a CloudMatrix supernode cluster without accessing directly the driver libraries, we simplify the modelling and assume that each node connects 16 devices independently of NIC. This is coherent with our hierarchical modelling, and does not invalidate our argument.

We tested the hierarchical *AllGather* using the existing global *AllGather* as a baseline. Each experiment is parameterized by the number of nodes involved in the collective operation, and the size of the shard gathered per worker. We used up to 8 nodes, and the values for the input size, in megabytes (MB), are in the set $\{1, 2, 4, 8, 16, 32, 64, 128\}$. To obtain the execution time for each test, we perform 10 warm-up rounds and 30 measurement rounds. We aggregate the 30 measurements per device using the maximum, and then we return the average value. Then, for each test, we divided the execution time by the input size per worker to obtain a normalized rate (seconds per megabyte), and aggregated it for each number of nodes using the harmonic mean. The comparative speedup gained can be seen in Figure 3.

The case in which we use one node is degenerate: the inter-node *AllGather* only introduces system overhead and the intra-node *AllGather* is equivalent to the baseline communication algorithm. The rest of the cases show significant speedup, especially when the number of nodes used is not a power of two. This could be a consequence of an implementation of the *AllGather* collective based on trees or recursive doubling. In both cases, the cost of an *AllGather* of local message size m, latency α, inverse bandwidth β and number of processors P

[1] HyperParallel open-source code: https://atomgit.com/mindspore/hyper-parallel

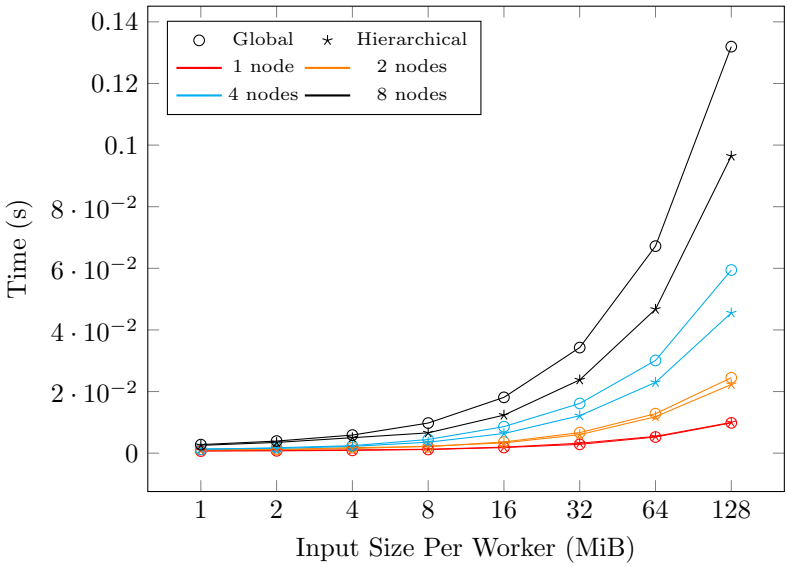


Fig. 3 Speedups achieved in each test, by number of nodes. The implementation achieves better general performance if the bar is over the dotted line, and worse performance otherwise. Node numbers that are power of two are highlighted and connected by segments.

can be modelled as $\alpha \cdot k_1 \cdot \lceil log_2(P) \rceil + k_2 \cdot P \cdot m \cdot \beta$. The speedup that we can obtain with this implementation, having 16 devices per node, is:

$$\frac{AllGather(P = 16 \cdot n,\ m = m_w, \alpha, \beta)}{AllGather(P = n,\ m = m_w, \alpha, \beta_1) + AllGather(P = 16,\ m = n \cdot m_w, \alpha, \beta_2)}$$

$$= \frac{\alpha \cdot k_1 \cdot \lceil log_2(16 \cdot n) \rceil + k_2 \cdot 16 \cdot n \cdot m_w \cdot \beta}{(\alpha \cdot k_1 \cdot \lceil log_2(n) \rceil + k_2 \cdot n \cdot m_w \cdot \beta_1) + (\alpha \cdot k_1 \cdot \lceil log_2(16) \rceil + k_2 \cdot 16 \cdot n \cdot m_w \cdot \beta_2)}$$

$$= \frac{\alpha \cdot k_1 \cdot \lceil log_2(16 \cdot n) \rceil + k_2 \cdot n \cdot m_w \cdot 16 \cdot \beta}{\alpha \cdot k_1 \cdot \lceil log_2(16 \cdot n) \rceil + k_2 \cdot n \cdot m_w \cdot (\beta_1 + 16 \cdot \beta_2)}$$

$$= \frac{A(n) + (K_2 \cdot 16 \cdot \beta) \cdot n}{A(n) + (K_2 \cdot (\beta_1 + 16 \cdot \beta_2)) \cdot n}$$

If the baseline's bandwidth is the minimum bandwidth of the network ($\beta = \beta_1$) and β_2 is negligible in comparison with β_1, the speedup tends to 16x, the number of devices per node, as the number of nodes grows. Fitting the unknown constants, it could be possible to obtain the real speedup in the case of CloudMatrix384, for which $n \leq 48$. The expression that we obtained for n matches the segmented line of Figure 3.

5.2 End-to-end training validation

To evaluate the performance gain that our proposal gives on real case scenarios, we compared our hierarchical *AllGather* implementation with the baseline version as the collective that unshards parameters when using FSDP for supervised training of large language models.

We chose Qwen3-VL-30B-A3B-Instruct [4], Qwen3.5-35B-A3B [26] and GLM-5 [3] as our target models, since they exhibit modern model architectures and are of interest in the industry. All of them show a sparse MoE design. Qwen3-VL processes tokens with conventional GroupQueryAttention layers. Qwen3.5 extends this by using both full attention and linear attention layers. GLM-5 only processes text input, but its hidden dimension is much larger, and uses KV compression and sparse token selection in its attention component. We train using a loop that contains a forward pass, a backward pass and an AdamW weight update, using *bfloat16* precision for parameters, *float16* for output and *float32* precision for gradient reductions and optimizer states. Our dataset is synthetic and each sample has a fixed sequence length of 1024. The batch size was adjusted for every model, in order to maximize the throughput of each worker without breaking the memory limit. For Qwen3.5-35B-A3B, the batch size per worker is set to 1, while for Qwen3-VL-30B-A3B-Instruct and GLM-5 it is set to 16.

We use 1, 2, 4 and 8 nodes in the CloudMatrix384 cluster, with 16, 32, 64 and 128 workers respectively, to measure the weak scaling that FSDP can obtain, with and without our optimization. In each of these eight training experiments per model, we measured step time in seconds, and peak memory allocated in gigabytes (GB). For each configuration, 20 training steps were measured. We discarded the first 5 measurements as warm-ups, and then reported the arithmetic average among the rest of the values. Our memory measurement remained constant for each configuration, and accounts for the maximum allocated memory during the step, including possible fragmentation. In Figure 4 we show the results of our experiments.

All experiments show a decrease in the average step time, validating our hypothesis. The memory measurements show little overhead with respect to the baseline. This is an artifact of our implementation: we use pinned-memory buffers to store intermediate results between successive data redistributions. With access to the low-level accelerator library, it is possible to alias intermediate buffers onto the memory region of the final result, so we consider this a removable artifact of our prototype.

The effect of hierarchical communication in the training of each model is variable, as the speedup chart in Figure 5 shows. Those models in which communication time plays a bigger part or cannot be overlapped with computation time display a bigger speedup. For Qwen3.5, the batch size used was very small, and non-overlapped communication is substantial. The speedups obtained follow a linear trend, with respect to the number of nodes, which is coherent with our formalization of the communication costs. The training of Qwen3-VL-30B is also greatly improved by our contribution, reaching a speedup of 1.204x

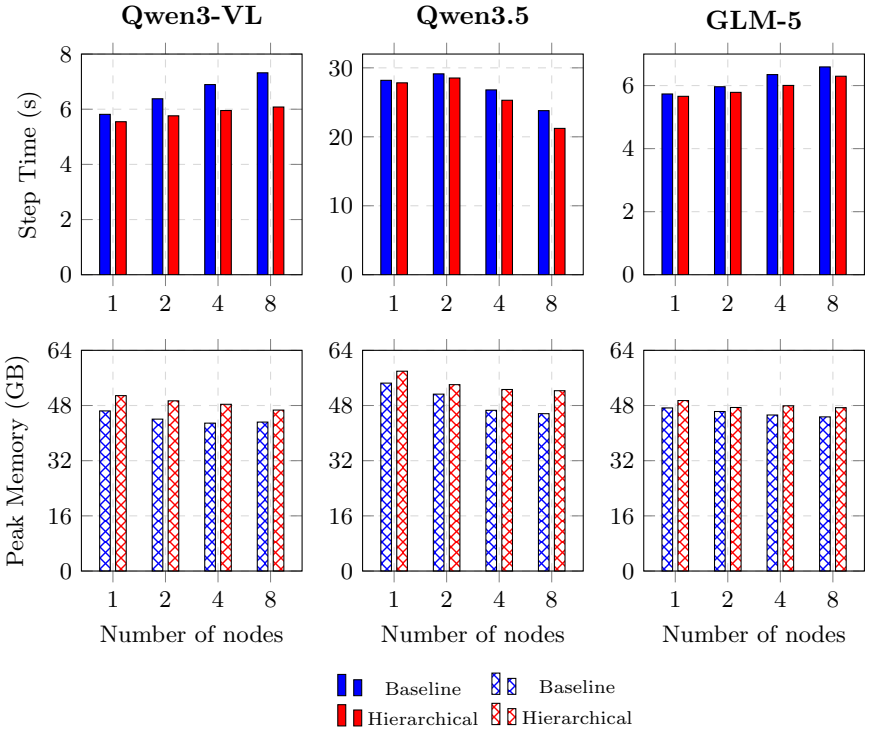


Fig. 4 Metrics obtained for the end-to-end validation tests. Top plots show the average step time, and the bottom plots show the peak memory usage. Each column corresponds to a different model. At each plot, the ticks are the number of nodes used to distribute the computations.

using eight nodes. However, the batch size used is higher, and the speedup does not follow the linear trend since communication might be hidden within the overlap. GLM-5's experts are an order of magnitude bigger than the other two models, in terms of parameter count, and the parameter unsharding represents a small fraction of the step time, compared with other communications and computations. Thus, the speedup achieved is smaller.

6 Related work

The communication overhead introduced by data redistribution in distributed deep learning has been identified as a critical bottleneck [15]. Recent surveys of automatic parallelism in DNN training [27] identify topology-aware communication as one of the foremost open challenges.

The complexity of choosing parallelism strategies has motivated a series of systems that automate this search. Jia et al. [28] first enumerated hidden dimensions of parallelism in CNNs, and later generalized this idea into the SOAP [29] search space of hybrid data/model parallelism. Alpa [30] unified intra- and inter-operator parallelism search into a single hierarchical

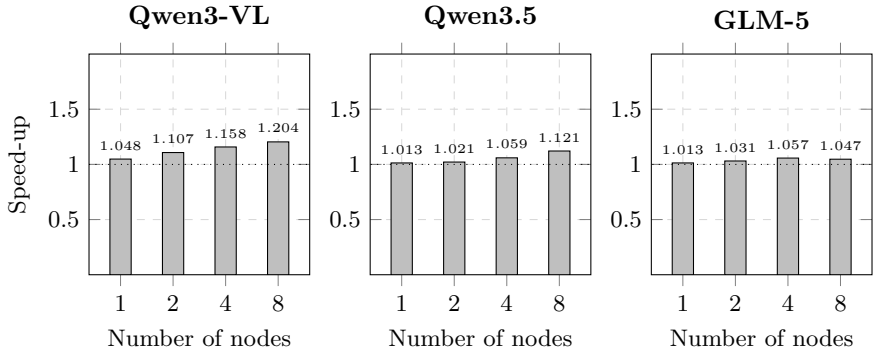


Fig. 5 Speedup of the training of different models, using a variable number of nodes.

optimization, using an integer linear program to jointly optimize computation and communication costs. For instance, their authors already consider the substitution of *AllReduce* for a combination of *ReduceScatter* + *AllGather*, to reduce communication overhead. TensorOpt [31] extended this line with a cost-model-based search over the strategy space for large-scale clusters. D-Rec [32] proposed an efficient strategy generation algorithm that scales to large networks, and SMSG [33] introduced a profiling-free parallelism model that avoids runtime measurements. These systems assume a uniform communication substrate and leave redistribution ordering to the runtime.

Approaches that focus on optimizing communication primitives have succeeded at synthesizing topology-aware algorithms for communication. A well-known example is SCCL [34], a collective synthesis solution that generates optimal algorithms using a cost model based on latency and bandwidth, and encodes them as solutions of SMT formulas. It proved useful to find optimized algorithms that beat the existing theoretical baseline for the NVIDIA DGX intranode topology. TACCL [35] extends the idea to generate arbitrary collectives, using communication sketches from the user as input and solving a series of mixed integer linear programs.

Finally, as close related work, we shall cite the efforts of Xie et al. [36] and Rink et al. [37]. The former proposes a hierarchy-aware algorithm based on profiled communication speeds rather than symbolic models, that generates candidate redistribution sequences and ranks them empirically. The latter builds communication semantics over distributed arrays, rather than using topological information, and minimizes the data exchanged during redistribution under the assumption of uniform communication speeds, proving optimality results in that restricted setting.

7 Conclusion

In this paper, we presented an algorithm that generates efficient redistribution strategies for distributed deep learning workloads executed on clusters with hierarchical topologies. Guided by a symbolic high-level cost model, the

algorithm decides which collective communications to perform and in which order. By analyzing the cost incurred on the communication overhead of each operator's redistributions, we offer a complementary vision to the analysis of parallel strategies of DNNs. We applied our algorithm to the study of the distributed training of state-of-the-art language models and observed a speedup in a hierarchical scenario compared to the baseline offered by an open-source AI framework.

Acknowledgements

This work was partially supported by the National Key R&D Program of China (Grant No.2023ZD0120400), and is also supported by the MindSpore open-source project with Huawei Technologies. Hector Buffière was an intern at Huawei and Ruben Coll Sanchez is an apprentice at Huawei.

Author Contributions

Hector Buffière, Pierre Leca and Chong Li initialize this work and established the original algorithms; Ruben Coll Sanchez, Hongxing Wang and Chong Li extend this work with a better algorithm, implemented on HyperParallel, and applied on SoTA DNNs and supernode hardware; all authors wrote the main manuscript text and reviewed the manuscript.

Conflict of interest

The authors declare no competing interests.

References

[1] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., *et al.*: Language models are few-shot learners. In: Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901 (2020)

[2] OpenAI: GPT-5.6: Frontier Intelligence That Scales with Your Ambition. https://openai.com/index/gpt-5-6/ Accessed 2026-08-30

[3] GLM-5-Team, Zeng, A., Lv, X., Hou, Z., Du, Z., Zheng, Q., Chen, B., Yin, D., Ge, C., Huang, C., Xie, C., Zhu, C., Yin, C., Wang, C., Pan, G., Zeng, H., Zhang, H., Wang, H., Chen, H., Zhang, J., et al.: GLM-5: from Vibe Coding to Agentic Engineering (2026). https://arxiv.org/abs/2602.15763

[4] Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Chen, Z., Cheng, L., Deng, W., Ding, G., Gao, C., Ge, C., Ge, W., Guo, Z., Huang, Q., Huang, J., Huang, F., Hui, B., Jiang, S., Li, Z., Li, M., Li, M., Li, K., Lin, Z., Lin, J., Liu, X., Liu, J., Liu, C., Liu, Y., Liu, D., Liu, S., Lu, D., Luo, R., Lv,

C., Men, R., Meng, L., Ren, X., Ren, X., Song, S., Sun, Y., Tang, J., Tu, J., Wan, J., Wang, P., Wang, P., Wang, Q., Wang, Y., Xie, T., Xu, Y., Xu, H., Xu, J., Zhang, B., Zheng, H., Zhong, J., Zhou, J., Zhou, Y., Zhu, K., Zhu, Y.: Qwen3-VL Technical Report (2025). https://arxiv.org/abs/2511.21631

[5] Jouppi, N.P., Lakshmanamurthy, S., Young, C., Patterson, D.: Google's Training Supercomputers From TPUv2 to Ironwood: Architectural Stability, Scale, Resilience, Power Efficiency, and Sustainability Across Five Generations. IEEE Micro **46**(4), 27–37 (2026). https://doi.org/10.1109/mm.2026.3699647

[6] Zuo, P., Lin, H., Deng, J., Zou, N., Yang, X., Diao, Y., Gao, W., Xu, K., Chen, Z., Lu, S., Qiu, Z., Li, P., Chang, X., Yu, Z., Miao, F., Zheng, J., Li, Y., Feng, Y., Wang, B., Zong, Z., Zhou, M., Zhou, W., Chen, H., Liao, X., Li, Y., Zhang, W., Zhu, P., Wang, Y., Xiao, C., Liang, D., Cao, D., Liu, J., Yang, Y., Bai, X., Li, Y., Xie, H., Wu, H., Yu, Z., Chen, L., Liu, H., Ding, Y., Zhu, H., Xia, J., Xiong, Y., Yu, Z., Liao, H.: Serving Large Language Models on Huawei CloudMatrix384 (2025). https://arxiv.org/abs/2506.12708

[7] NVIDIA: NVIDIA Vera Rubin NVL72. https://www.nvidia.com/en-us/data-center/vera-rubin-nvl72/ Accessed 2026-08-30

[8] NVIDIA: NVLink & NVLink Switch: Fastest HPC Data Center Platform. https://www.nvidia.com/en-us/data-center/nvlink/ Accessed 2026-08-30

[9] Li, B.: OpenURMA: A Clean-Room Open Implementation of the Unified Bus Protocol (2026). https://arxiv.org/abs/2605.28717

[10] Dean, J., Corrado, G., Monga, R., Chen, K., Devin, M., Mao, M., Ranzato, M., Senior, A., Tucker, P., Yang, K., et al.: Large scale distributed deep networks. Advances in Neural Information Processing Systems **25** (2012)

[11] Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., Catanzaro, B.: Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv preprint (2020) 1909.08053

[12] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., *et al.*: Pytorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems, vol. 32 (2019)

[13] Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., *et al.*: TensorFlow: A System for Large-Scale Machine Learning. In: Proceedings of the 12th USENIX

Conference on Operating Systems Design and Implementation. OSDI '16, pp. 265–283 (2016)

[14] Tong, Z., Du, N., Song, X., Wang, X.: Study on MindSpore Deep Learning Framework. In: 2021 17th International Conference on Computational Intelligence and Security. CIS, pp. 183–186 (2021)

[15] Zhang, Z., Chang, C., Lin, H., Wang, Y., Arora, R., Jin, X.: Is network the bottleneck of distributed training? In: Proceedings of the Workshop on Network Meets AI & ML. NetAI '20, pp. 8–13 (2020)

[16] Pinedo, M.L.: Scheduling: Theory, Algorithms, and Systems. Springer (2022)

[17] Clos, C.: A study of non-blocking switching networks. Bell System Technical Journal **32**(2), 406–424 (1953)

[18] Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C.-C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., Desmaison, A., Balioglu, C., Damania, P., Nguyen, B., Chauhan, G., Hao, Y., Mathews, A., Li, S.: PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. Proceedings of the VLDB Endowment **16**(12), 3848–3859 (2023)

[19] Zhang, X., Sun, B., Su, T., Zhang, Q., Bao, C., Chen, L., Jin, X.: HyperParallel: A Supernode-Affinity AI Framework (2026). https://arxiv.org/abs/2603.03731

[20] Zhang, D., Song, J., Bi, Z., Song, X., Yuan, Y., Wang, T., Yeong, J., Hao, J.: Mixture of Experts in Large Language Models (2025). https://arxiv.org/abs/2507.11181

[21] Jacobs, S.A., Tanaka, M., Zhang, C., Zhang, M., Song, S.L., Rajbhandari, S., He, Y.: DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models (2023). https://arxiv.org/abs/2309.14509

[22] Message Passing Interface Forum: MPI: A Message-Passing Interface Standard Version 5.0. (2025). https://www.mpi-forum.org/docs/mpi-5.0/mpi50-report.pdf

[23] Li, C., Hains, G.: A simple bridging model for high-performance computing. In: 2011 International Conference on High Performance Computing & Simulation. HPCS, pp. 249–256 (2011)

[24] Li, C., Hains, G.: GPS: Towards Simplified Communication on SGL Model. In: 2014 IEEE International Parallel & Distributed Processing Symposium Workshops, pp. 727–736 (2014)

[25] Rajbhandari, S., Rasley, J., Ruwase, O., He, Y.: ZeRO: Memory optimizations toward training trillion parameter models. In: SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–16 (2020)

[26] Qwen Team: Qwen3.5-35B-A3B. https://huggingface.co/Qwen/Qwen3.5-35B-A3B Accessed 2026-08-30

[27] Liang, P., Tang, Y., Zhang, X., Bai, Y., Su, T., Lai, Z., Li, D.: A survey on auto-parallelism of neural networks training. International Journal of Parallel Programming **52**(2), 251–285 (2024)

[28] Jia, Z., Lin, S., Qi, C.R., Aiken, A.: Exploring hidden dimensions in parallelizing convolutional neural networks. In: Proceedings of the 35th International Conference on Machine Learning, pp. 2279–2288 (2018)

[29] Jia, Z., Zaharia, M., Aiken, A.: Beyond data and model parallelism for deep neural networks. Proceedings of Machine Learning and Systems **1**, 1–13 (2019)

[30] Zheng, L., Li, Z., Zhang, H., Zhuang, Y., Chen, Z., Huang, Y., Wang, Y., Xu, Y., Zhuo, D., Gonzalez, J.E., *et al.*: Alpa: Automating inter-and intra-operator parallelism for distributed deep learning. In: 16th USENIX Symposium on Operating Systems Design and Implementation, pp. 559–578 (2022)

[31] Cai, Z., Yan, X., Ma, K., Wu, Y., Huang, Y., Cheng, J., Su, T., Yu, F.: TensorOpt: Exploring the Tradeoffs in Distributed DNN Training With Auto-Parallelism. IEEE Transactions on Parallel and Distributed Systems **33**(8), 1967–1981 (2022)

[32] Wang, H., Li, C., Tachon, T., Wang, H., Yang, S., Limet, S., Robert, S.: Efficient and systematic partitioning of large and deep neural networks for parallelization. In: European Conference on Parallel Processing. Euro-Par, pp. 201–216 (2021)

[33] Wang, H., Tachon, T., Li, C., Robert, S., Limet, S.: SMSG: Profiling-Free Parallelism Modeling for Distributed Training of DNN. International Journal of Parallel Programming, 1–19 (2022)

[34] Cai, Z., Liu, Z., Maleki, S., Musuvathi, M., Mytkowicz, T., Nelson, J., Saarikivi, O.: Synthesizing optimal collective algorithms. In: Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. PPoPP '21, pp. 62–75 (2021)

[35] Shah, A., Chidambaram, V., Cowan, M., Maleki, S., Musuvathi, M., Mytkowicz, T., Nelson, J., Saarikivi, O., Singh, R.: TACCL: Guiding

collective algorithm synthesis using communication sketches. In: 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23), pp. 593–612. USENIX Association, Boston, MA (2023)

[36] Xie, N., Norman, T., Grewe, D., Vytiniotis, D.: Synthesizing optimal parallelism placement and reduction strategies on hierarchical systems for deep learning. In: Proceedings of Machine Learning and Systems, vol. 4, pp. 548–566 (2022)

[37] Rink, N.A., Paszke, A., Vytiniotis, D., Schmid, G.S.: Memory-efficient array redistribution through portable collective communication. arXiv preprint (2021) 2112.01075